

PROJECT PROPOSAL
**DESIGN AND IMPLEMENTATION OF HANGMAN
GAME USING ASSEMBLY LANGUAGE**

SUBMITTED BY:

Team Members:

1. Abdullah Ejaz Janjua (2023038)
2. Muhammad Ahmad Amjad (2023361)
3. Muhammad Haris (2023428)

SUMITTED TO:

Instructor:

Sir Zaheer Ahmed
FCSE
CE – 222 Lab

OUTLINE:

DESIGN DECISIONS AND TRADE-OFFS.....	Pg.no:3
IMPLEMENTATION CHALLENGES AND SOLUTIONS.....	Pg.no:4
TESTING AND VERIFICATION METHODS	Pg.no:7
RESULTS AND CONCLUSIONS	Pg.no.7

EXECUTIVE SUMMARY

This report details the implementation of a Hangman game written in x86 Assembly language. The program demonstrates fundamental assembly programming concepts while delivering an interactive text-based game experience. The implementation includes difficulty levels, visual ASCII art representation of the hangman, score tracking, and comprehensive input validation.

1. DESIGN DECISIONS AND TRADE-OFFS

1.1 MEMORY MANAGEMENT

Design Decision: The program uses a segmented memory model with separate data and code segments.

- The `.model small` directive was chosen to balance memory usage with program requirements.
- The `.stack 100h` directive allocates a 256-byte stack, sufficient for the program's procedure calls and data storage needs.

Trade-off: While a larger memory model would have allowed for more extensive word lists and features, the small model optimizes memory usage for this application's scale.

1.2 DATA STRUCTURES

Design Decision: Several key data structures were implemented:

- Word lists organized by difficulty levels with null-terminated strings
- Word pointer arrays (`easyPtrs`, `mediumPtrs`, `hardPtrs`) for efficient random word selection
- Buffer arrays for game state (`guesses_wrd`, `entered_words`)
- Pre-defined ASCII art for visual representation of hangman stages

Trade-off: Using fixed-size arrays simplified memory management but limited the length of words and the number of guesses that could be tracked.

1.3 MACRO IMPLEMENTATION

Design Decision: Extensive use of macros for common operations:

- `display_str` - Simplifies string output with newlines
- `delay` - Creates timing delays for better user experience
- `get_len` - Calculates string length
- `add_null` - Adds string termination character
- `print_guess_word` - Displays the current state of the guessed word
- `print_var` - Handles variable printing (converting numeric values to displayable characters)

- `cls` - Clears the screen

Trade-off: Macros increase code size but significantly improve readability and maintainability. The inline expansion of macros trades memory efficiency for execution speed.

1.4 USER INTERFACE

Design Decision: Text-based UI with ASCII art visualization:

- Multiple hangman stages (stage0 through stage6)
- Clear screen between major game state changes
- Display of score and remaining chances
- Hint system that reveals masked letters in the word

Trade-off: ASCII art takes up significant memory but substantially improves user engagement compared to purely textual output.

1.5 DIFFICULTY LEVELS

Design Decision: Three difficulty levels with appropriate word sets:

- Easy: short, common words (apple, cat, india, dog)
- Medium: moderate length words (banana, tiger, france, lion)
- Hard: longer, more complex words (cherry, elephant, australia, rhinoceros)

Trade-off: Limited word selection per difficulty level to conserve memory, trading variety for implementation simplicity.

2. IMPLEMENTATION CHALLENGES AND SOLUTIONS

2.1 RANDOM WORD SELECTION

Challenge: Implementing randomness in Assembly without built-in random functions.

Solution: Used system clock ticks (INT 1Ah) as a pseudo-random seed:

```
; Get clock ticks
mov ah, 00h
int 1Ah
mov al, dl
and al, 00000111b ; Mask to have only 7 possibilities
cmp al, 04
jbe ok_index ; If selected index < 4
mov al, 03 ; Fallback
```

This approach provides sufficient randomization for game purposes while being resource-efficient.

2.2 STRING MANIPULATION

Challenge: Working with strings in Assembly without high-level string functions.

Solution: Implemented custom procedures and macros for string operations:

- `get_len` to calculate string length
- Custom string copying in the `get_string` procedure
- String comparison in `compare_and_store_word`

This custom approach allowed precise control over string operations at the cost of additional implementation complexity.

2.3 INPUT VALIDATION AND DUPLICATE DETECTION

Challenge: Tracking letters already guessed and preventing duplicates.

Solution: Created the `is_entered` procedure that:

1. Searches through a buffer of previously entered characters
2. Returns status codes (0: new letter, 1: duplicate, 2: buffer full)
3. Stores new letters in the buffer

`is_entered PROC`

```
mov si, offset entered_words
```

```
search_loop:
```

```
mov cl, [si]
```

```
cmp cl, al
```

```
je duplicate_found
```

```
cmp cl, '$'
```

```
je not_found
```

```
inc si
```

```
jmp search_loop
```

```
; ...
```

`is_entered ENDP`

2.4 GAME STATE MANAGEMENT

Challenge: Keeping track of remaining letters, score, and game progression.

Solution:

- Used the `len_wrd` variable to track remaining unguessed letters
- Decrement chances for incorrect guesses
- Implemented `end_game` procedure to check win conditions

`end_game PROC`

`add score, 2`

`sub len_wrd, 1`

`cmp len_wrd, 0`

`jne skip`

`jmp game_over`

`skip:`

`RET`

`end_game ENDP`

2.5 NUMBER DISPLAY

Challenge: Converting numeric values to displayable text.

Solution: Implemented the `print_var` macro that:

1. Divides the number by 10 to separate tens and ones
2. Converts each digit to its ASCII representation (adding 48)
3. Displays each digit individually

`print_var MACRO var`

`LOCAL skip_tens`

`push ax`

`push bx`

`push dx`

`mov al, var`

`mov ah, 0`

```
mov bl, 10
```

```
div bl ; AL = quotient (tens), AH = remainder (ones)
```

```
ENDM
```

3. TESTING AND VERIFICATION METHODS

3.1 INCREMENTAL TESTING APPROACH

The program was tested using an incremental approach:

1. **Core Mechanics Testing:** First verified basic word selection, letter guessing, and game state updates
2. **Input Validation Testing:** Example Duplicate letter detection
3. **Visual Element Testing:** Verified ASCII art displays at appropriate game stages
4. **Game Flow Testing:** Confirmed the logical progression through the entire game

3.2 EDGE CASE TESTING

Several edge cases were specifically tested:

- **Word Selection:** Tested words of varying lengths from each difficulty level
- **Input Handling:** Tested with repeated letters, and buffer overflow scenarios
- **Game State Transitions:** Verified correct transitions between win and lose states

3.3 REGISTER PRESERVATION

Special attention was paid to register preservation in procedures:

- Critical registers (AX, BX, DX) are properly pushed and popped in macros and procedures
- Care was taken to preserve values across procedure calls that might modify the same registers

3.4 MEMORY INTEGRITY

Testing included verification of memory integrity:

- Buffer boundaries are respected with proper string termination
- Arrays are properly initialized and accessed within bounds
- No unintended memory overwrites occur during string operations

4. RESULTS AND CONCLUSIONS

4.1 PERFORMANCE ANALYSIS

The Hangman game implementation demonstrates efficient assembly language programming with:

- Minimal memory footprint due to the small memory model
- Fast execution due to direct hardware access

- Efficient input/output operations using DOS interrupts
- Resource-efficient string manipulation without external libraries

4.2 CODE STRUCTURE ASSESSMENT

The code structure shows thoughtful organization:

- Clear separation of data and code segments
- Well-defined procedures for distinct functional areas
- Effective use of macros to improve readability
- Careful register management and preservation

4.3 USER EXPERIENCE EVALUATION

The game provides a complete and engaging user experience:

- Multiple difficulty levels for variable challenge
- Visual representation of game progress through ASCII art
- Feedback on correct/incorrect guesses
- Score tracking and final score calculation
- Clear win/lose conditions and game endings

4.4 LIMITATIONS AND POTENTIAL IMPROVEMENTS

Despite the successful implementation, several limitations exist:

1. **Limited Word Set:** Each difficulty level contains only 4 words, which limits replay value
2. **Fixed Buffer Sizes:** The program uses fixed-size arrays that could overflow with certain inputs
3. **Basic Error Handling:** Some edge cases in user input might not be fully handled
4. **DOS Platform Dependency:** The implementation relies on DOS interrupts, limiting portability

Potential improvements include:

- Expanding the word sets for each difficulty level
- Implementing dynamic memory allocation for buffers
- Adding more sophisticated input validation
- Enhancing the scoring system with time-based components
- Adding sound effects using PC speaker interrupts

4.5 FINAL CONCLUSION

This Assembly language implementation of Hangman successfully demonstrates:

1. Low-level system programming concepts including memory management, register manipulation, and interrupt usage
2. Effective algorithmic implementation in Assembly
3. User interface design within the constraints of text-based environments

4. **Game logic implementation without high-level language abstractions**

The project achieves its goals while highlighting both the challenges and capabilities of Assembly language programming. Despite the inherent constraints of the platform and language, the implementation delivers a functional and engaging gaming experience that effectively balances computational efficiency with user enjoyment.

APPENDIX: KEY ASSEMBLY CONCEPTS DEMONSTRATED

1. **Memory Segmentation:** Use of data, code, and stack segments
2. **Interrupt Usage:** DOS and BIOS interrupt calls for I/O and system functions
3. **Macro Programming:** Creation and use of reusable code blocks
4. **Procedure Implementation:** Function-like encapsulation with parameter passing
5. **String Processing:** Low-level character manipulation and comparison
6. **Control Structures:** Implementation of loops and conditional branching
7. **Register Management:** Efficient use of CPU registers for computation
8. **ASCII Art Rendering:** Display of graphical representations using text characters